

BTA 522 – Project

AUTHOR

Kade Kenzler

r one time console installs

```
####install.packages(c("arrow", "readr", "readxl", "writexl", "dplyr", "ggplot2", "lessR", "purrr", "stringr",
"lubridate"))
```

“global” variables

file/directory_path(s)

```
epaOZ_dailyDir <- "C:/Users/kadek/OneDrive/1Drive - Documents/Education/Classes/masters/fall125/
epaPM_dailyDir <- "C:/Users/kadek/OneDrive/1Drive - Documents/Education/Classes/masters/fall125/
epa_sitesPath <- "C:/Users/kadek/OneDrive/1Drive - Documents/Education/Classes/masters/fall125/4
noaa_pm25Dir <- "C:/Users/kadek/OneDrive/1Drive - Documents/Education/Classes/masters/fall125/4
featherDir <- "C:/Users/kadek/Documents/Quarto/kkenzler bta522 project_files"
```

user input details

```
# editable state set
state_set <- c(
  "Washington", "California",    # west
  "Minnesota",  "Texas",        # central
  "New York",   "Florida",       # east
  "Colorado",   "Georgia"       # extra coverage
)

# pick a start date - 2014-01-01 is start of original data - the upper bound will be whatever t
start_date <- as.Date("2014-01-01")
```

libraries / packages

```
library(arrow)
library(readr)
library(readxl)
library(writexl)
library(dplyr)
library(ggplot2)
library(lessR)
```

```
library(tibble)
library(purrr)
library(stringr)
library(lubridate)
```

main project body start

describing the data

The dataset integrates multi-year environmental and weather observations from two major U.S. sources: the EPA's Air Quality System (AQS) and the NOAA's Global Historical Climatology Network (GHCN-Daily). The EPA daily and annual files provide pollutant concentration data—principally ozone (44201) and PM2.5 (88101)—recorded across hundreds of monitoring sites nationwide, with attributes such as geographic codes, site coordinates, units of measure, and sampling duration. The NOAA component adds corresponding meteorological variables (e.g., precipitation, temperature, and snow depth) for multiple weather stations identified by USW codes. Together, these datasets allow cross-analysis of air quality and weather patterns at the regional or state level.

reading data into dfs

```
# csv reader helper function
read_all_csv <- function(folder, pattern_regex) {
  # recursively read all CSV files under `folder` whose path matches `pattern_regex`,
  # tag each with a source_file column, and return a single combined tibble.
  csvs <- list.files(folder, pattern = "\\\\.csv$", recursive = TRUE, full.names = TRUE)
  csvs <- csvs[stringr::str_detect(csvs, pattern_regex)]
  if (length(csvs) == 0) return(tibble())
  purrr::map_dfr(
    csvs,
    ~ readr::read_csv(.x, show_col_types = FALSE) %>%
      dplyr::mutate(source_file = basename(.x))
  )
}

# load csvs
epaOZ_daily <- read_all_csv(epaOZ_dailyDir, "daily_44201_\\d{4}")
epaPM_daily <- read_all_csv(epaPM_dailyDir, "daily_88101_\\d{4}")
epa_sites <- read_csv(epa_sitesPath)
noaaWeather <- read_all_csv(noaa_pm25Dir, "USW_\\d{8}")
```

Codebook & Variable Description

The project's data codebook is derived by extending the output of the details() diagnostic function used

below. That function automatically enumerates every variable within each imported dataset, listing its storage type, number of non-missing and missing values, and the count of unique entries—a first-pass structural summary analogous to the metadata section of a formal codebook. Building on that output, the codebook elaborates the conceptual meaning of each key field (for example, “Arithmetic Mean” as the daily average pollutant concentration, or “Date Local” as the observation date converted to a true date object). In short, the `details()` tables form the empirical foundation for the codebook, while the written descriptions translate those technical fields into human-readable explanations of what each variable measures and how it will be used analytically.

displaying the dataframe(s)

row, index, column, type, value count, missing count, unique count

```
# df reader helper display function
details <- function(df, brief = TRUE) {
  if (is.null(df) || ncol(df) == 0) {
    cat("details(): data frame is empty (no columns loaded)\n")
    return(invisible(NULL))
  }
  dStats <- tibble::tibble(
    variable = names(df),
    type      = vapply(df, function(x) paste(class(x), collapse=","), character(1)),
    values     = vapply(df, function(x) sum(!is.na(x)), integer(1)),
    missing    = vapply(df, function(x) sum(is.na(x)), integer(1)),
    unique     = vapply(df, dplyr::n_distinct, integer(1))
  ) |>
  dplyr::mutate(index = dplyr::row_number() - 1) |>
  dplyr::select(index, dplyr::everything())
  print(dStats)
  if (!brief) {
    cat("\nRows with any missing values:\n\n")
    print(dplyr::filter(df, dplyr::if_any(dplyr::everything(), is.na)))
  }
}

# display dfs
details(epaOZ_daily)
```

A tibble: 30 × 6

	index	variable	type	values	missing	unique
	<dbl>	<chr>	<chr>	<int>	<int>	<int>
1	0	State Code	character	4319631	0	53
2	1	County Code	character	4319631	0	139
3	2	Site Num	character	4319631	0	332
4	3	Parameter Code	numeric	4319631	0	1
5	4	POC	numeric	4319631	0	5
6	5	Latitude	numeric	4319631	0	1546
7	6	Longitude	numeric	4319631	0	1547

```
 /      6 Longitude      numeric  4319631      0      1377
8      7 Datum           character 4319631      0        2
9      8 Parameter Name   character 4319631      0        1
10     9 Sample Duration  character 4319631      0        1
# i 20 more rows
```

```
details(epaPM_daily)
```

```
# A tibble: 30 × 6
  index variable      type      values missing unique
  <dbl> <chr>          <chr>    <int>  <int>  <int>
1     0 State Code    character 6735949      0     54
2     1 County Code   character 6735949      0    137
3     2 Site Num       character 6735949      0    326
4     3 Parameter Code numeric    6735949      0     1
5     4 POC            numeric    6735949      0    20
6     5 Latitude       numeric    6735949      0   1380
7     6 Longitude      numeric    6735949      0   1380
8     7 Datum          character 6735949      0     2
9     8 Parameter Name  character 6735949      0     1
10    9 Sample Duration character 6735949      0     3
# i 20 more rows
```

```
details(epa_sites)
```

```
# A tibble: 28 × 6
  index variable      type      values missing unique
  <dbl> <chr>          <chr>    <int>  <int>  <int>
1     0 State Code    character 20940      0     56
2     1 County Code   character 20940      0    244
3     2 Site Number    character 20940      0   1226
4     3 Latitude       numeric   20935      5   19369
5     4 Longitude      numeric   20936      4   19393
6     5 Datum          character 20940      0     4
7     6 Elevation      numeric   20940      0   2411
8     7 Land Use       character 19791    1149     11
9     8 Location Setting character 19923    1017      5
10    9 Site Established Date Date       20940      0   4181
# i 18 more rows
```

```
details(noaaWeather)
```

```
# A tibble: 187 × 6
  index variable      type      values missing unique
  <dbl> <chr>          <chr>    <int>  <int>  <int>
1     0 STATION       character 1581331      0     51
2     1 DATE            Date      1581331      0   66634
3     2 LATITUDE       numeric   1581331      0     51
4     3 LONGITUDE      numeric   1581331      0     51
```

```

5      4 ELEVATION      numeric 1581331      0      49
6      5 NAME          character 1581331      0      51
7      6 PRCP          numeric 1551167    30164    710
8      7 PRCP_ATTRIBUTES character 1551167    30164    77
9      8 SNOW          numeric 1477811   103520   223
10     9 SNOW_ATTRIBUTES character 1477811   103520    69
# i 177 more rows

```

pollutant tables: data cleaning & verifications

normalize/subset

```

# simplified/consistent pollutant tables attribute names
epaOZ_daily_clean <- epaOZ_daily |>
  dplyr::transmute(
    state_code = `State Code`,
    county_code = `County Code`,
    site_num = `Site Num`,
    parameter = `Parameter Code`,
    date = lubridate::ymd(`Date Local`),
    value = `Arithmetic Mean`,
    units = `Units of Measure`,
    aqs_site_id = paste(state_code, county_code, site_num, sep = "-"),
    source_file
  )

# simplified/consistent pollutant tables attribute names
epaPM_daily_clean <- epaPM_daily |>
  dplyr::transmute(
    state_code = `State Code`,
    county_code = `County Code`,
    site_num = `Site Num`,
    parameter = `Parameter Code`,
    date = lubridate::ymd(`Date Local`),
    value = `Arithmetic Mean`,
    units = `Units of Measure`,
    aqs_site_id = paste(state_code, county_code, site_num, sep = "-"),
    source_file
  )

# simplified/consistent site tables attribute names
epa_sites_clean <- epa_sites |>
  dplyr::transmute(
    state_code = `State Code`,
    county_code = `County Code`,
    site_num = `Site Number`,
    state_name = `State Name`,

```

```
    county_name = `County Name`,
    latitude    = Latitude,
    longitude   = Longitude,
    aqs_site_id = paste(state_code, county_code, site_num, sep = "-")
  )

# attaching site names to pollutant tables
epaOZ_daily_named <- epaOZ_daily_clean |>
  dplyr::left_join(epa_sites_clean, by = c("aqs_site_id", "state_code", "county_code", "site_num"))

epaPM_daily_named <- epaPM_daily_clean |>
  dplyr::left_join(epa_sites_clean, by = c("aqs_site_id", "state_code", "county_code", "site_num"))

sel_ozone <- epaOZ_daily_named |>
  dplyr::filter(state_name %in% state_set, date >= start_date)

sel_pm25 <- epaPM_daily_named |>
  dplyr::filter(state_name %in% state_set, date >= start_date)

# converting selected fields to categorical (factor) types
sel_ozone <- sel_ozone %>%
  mutate(
    state_name = factor(state_name),
    units      = factor(units),
    parameter  = factor(parameter)
  )

sel_pm25 <- sel_pm25 %>%
  mutate(
    state_name = factor(state_name),
    units      = factor(units),
    parameter  = factor(parameter)
  )
```

aggregating (& verification)

```
# aggregation of pollutant tables
ozone_state_daily <- sel_ozone |>
  dplyr::group_by(state_name, date) |>
  dplyr::summarise(ozone_ppm = mean(value, na.rm = TRUE), .groups = "drop")

pm25_state_daily <- sel_pm25 |>
  dplyr::group_by(state_name, date) |>
  dplyr::summarise(pm25_ugm3 = mean(value, na.rm = TRUE), .groups = "drop")

pollutants_state_daily <- pm25_state_daily |>
  dplyr::full_join(ozone_state_daily, by = c("state_name", "date")) |>
  dplyr::arrange(state_name, date)
```

```
# verification
check_units <- function(df, units_col = "units") {
  if (!units_col %in% names(df)) return(invisible(NULL))
  df |>
    dplyr::group_by(state_name) |>
    dplyr::summarise(n_units = dplyr::n_distinct(.data[[units_col]])) |>
    dplyr::filter(n_units > 1)
}

# quick check
check_units(sel_ozone)
```

```
# A tibble: 0 × 2
#   i 2 variables: state_name <fct>, n_units <int>
```

```
check_units(sel_pm25)
```

```
# A tibble: 0 × 2
#   i 2 variables: state_name <fct>, n_units <int>
```

```
dplyr::glimpse(pollutants_state_daily)
```

```
Rows: 32,144
Columns: 4
$ state_name <fct> California, California, California, California, California,...
$ date       <date> 2014-01-01, 2014-01-02, 2014-01-03, 2014-01-04, 2014-01-05...
$ pm25_ugm3  <dbl> 30.327717, 21.007516, 23.042277, 23.757560, 18.493957, 20.1...
$ ozone_ppm  <dbl> 0.02210794, 0.01939411, 0.01820570, 0.02279232, 0.02556394,...
```

noaa table(s): data cleaning & verifications

clean/subset

```
# map two-letter state abbreviations to full names
abbr_to_full <- c(
  AL = "Alabama", AK = "Alaska", AZ = "Arizona", AR = "Arkansas",
  CA = "California", CO = "Colorado", CT = "Connecticut", DE = "Delaware",
  FL = "Florida", GA = "Georgia", HI = "Hawaii", ID = "Idaho",
  IL = "Illinois", IN = "Indiana", IA = "Iowa", KS = "Kansas",
  KY = "Kentucky", LA = "Louisiana", ME = "Maine", MD = "Maryland",
  MA = "Massachusetts", MI = "Michigan", MN = "Minnesota", MS = "Mississippi",
  MO = "Missouri", MT = "Montana", NE = "Nebraska", NV = "Nevada",
  NH = "New Hampshire", NJ = "New Jersey", NM = "New Mexico", NY = "New York",
  NC = "North Carolina", ND = "North Dakota", OH = "Ohio", OK = "Oklahoma",
  OR = "Oregon", PA = "Pennsylvania", RI = "Rhode Island", SC = "South Carolina",
```

```
SD = "South Dakota", TN = "Tennessee", TX = "Texas", UT = "Utah",
VT = "Vermont", VA = "Virginia", WA = "Washington", WV = "West Virginia",
WI = "Wisconsin", WY = "Wyoming", DC = "District of Columbia"
)

# clean and derive state name from station name
noaa_clean <- noaaWeather |>
  dplyr::transmute(
    station = STATION,
    date     = as.Date(Date),
    name_raw = NAME,
    state_abbr = stringr::str_extract(NAME, "[A-Z]{2}(?=\s+US$)",
    state_name = abbr_to_full[state_abbr],
    prcp = as.numeric(PRCP),
    tmax = as.numeric(TMAX),
    tmin = as.numeric(TMIN)
  ) |>
  dplyr::filter(!is.na(state_name), date >= start_date)

# auto-detect temp (edits noaa_clean in place to c)
# if all temps are NA, skip conversion
non_na_tmax <- noaa_clean$tmax[!is.na(noaa_clean$tmax)]

if (length(non_na_tmax) > 0) {
  q <- stats::quantile(non_na_tmax, probs = c(0.1, 0.5, 0.9), na.rm = TRUE)
  med <- q[[2]]

  temp_mode <- dplyr::case_when(
    med >= 120 ~ "tenths_c", # e.g., 150 -> 15.0 °C
    med >= 60 & med <= 110 ~ "fahrenheit", # e.g., 86 °F
    med >= 5 & med <= 45 ~ "celsius", # e.g., 20 °C
    TRUE ~ "unknown"
  )
} else {
  temp_mode <- "unknown"
}

convert_to_c <- function(x, mode) {
  if (mode == "tenths_c") x / 10
  else if (mode == "fahrenheit") (x - 32) * 5/9
  else x # "celsius" or "unknown" => pass-through
}

# apply conversion; keep a column noting the applied
noaa_clean <- noaa_clean |>
  dplyr::mutate(
    tmax = convert_to_c(tmax, temp_mode),
    tmin = convert_to_c(tmin, temp_mode),
    temp_mode = temp_mode
  )
```


aggregating (& verification)

```
noaa_state_daily <- noaa_clean |>
  dplyr::group_by(state_name, date) |>
  dplyr::summarise(
    prcp = mean(prcp, na.rm = TRUE),
    tmax = mean(tmax, na.rm = TRUE),
    tmin = mean(tmin, na.rm = TRUE),
    .groups = "drop"
  )

# quick check
unique(noaa_clean$temp_mode)
```

```
[1] "tenths_c"
```

```
summary(noaa_state_daily$tmax)
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.	NA's
-24.9	11.7	21.1	19.6	28.9	48.9	45

```
dplyr::glimpse(noaa_state_daily)
```

```
Rows: 206,209
```

```
Columns: 5
```

```
$ state_name <chr> "Alabama", "Alabama", "Alabama", "Alabama", "Alabama", "Ala...
$ date       <date> 2014-01-01, 2014-01-02, 2014-01-03, 2014-01-04, 2014-01-05...
$ prcp       <dbl> 0, 76, 0, 0, 28, 0, 0, 0, 0, 8, 175, 0, 76, 10, 3, 0, 0, 0,...
$ tmax       <dbl> 11.7, 10.6, 2.8, 8.9, 14.4, -1.0, -4.3, 7.2, 13.9, 13.9, 18...
$ tmin       <dbl> -0.5, -3.8, -7.1, -1.6, -1.0, -11.6, -13.8, -9.9, -2.7, 6.7...
```

merge & align

```
# merge daily state-level pollutants with NOAA weather
merged_daily <- pollutants_state_daily |>
  dplyr::left_join(noaa_state_daily, by = c("state_name", "date")) |>
  dplyr::arrange(state_name, date)

# quick sanity checks
merged_daily |>
  dplyr::group_by(state_name) |>
  dplyr::summarise(
    n_rows = dplyr::n(),
    min_date = min(date, na.rm = TRUE),
```

```

max_date = max(date, na.rm = TRUE),
pm25_na  = sum(is.na(pm25_ugm3)),
o3_na    = sum(is.na(ozone_ppm)),
prcp_na  = sum(is.na(prcp)),
tmax_na  = sum(is.na(tmax)),
tmin_na  = sum(is.na(tmin))
) |>
dplyr::arrange(state_name) |>
print(n = 100)

```

A tibble: 8 × 9

	state_name	n_rows	min_date	max_date	pm25_na	o3_na	prcp_na	tmax_na	tmin_na
	<chr>	<int>	<date>	<date>	<int>	<int>	<int>	<int>	<int>
1	California	4018	2014-01-01	2024-12-31	0	0	0	20	21
2	Colorado	4018	2014-01-01	2024-12-31	0	0	0	16	18
3	Florida	4018	2014-01-01	2024-12-31	0	0	0	0	0
4	Georgia	4018	2014-01-01	2024-12-31	0	0	0	0	0
5	Minnesota	4018	2014-01-01	2024-12-31	0	0	0	0	0
6	New York	4018	2014-01-01	2024-12-31	0	0	0	0	0
7	Texas	4018	2014-01-01	2024-12-31	0	0	0	0	0
8	Washington	4018	2014-01-01	2024-12-31	0	0	4	0	1

generating the feather file

```

arrow::write_feather(
  merged_daily,
  file.path(featherDir, "10-28-25 OZ PM TMP merge 2014+.feather")
)

```

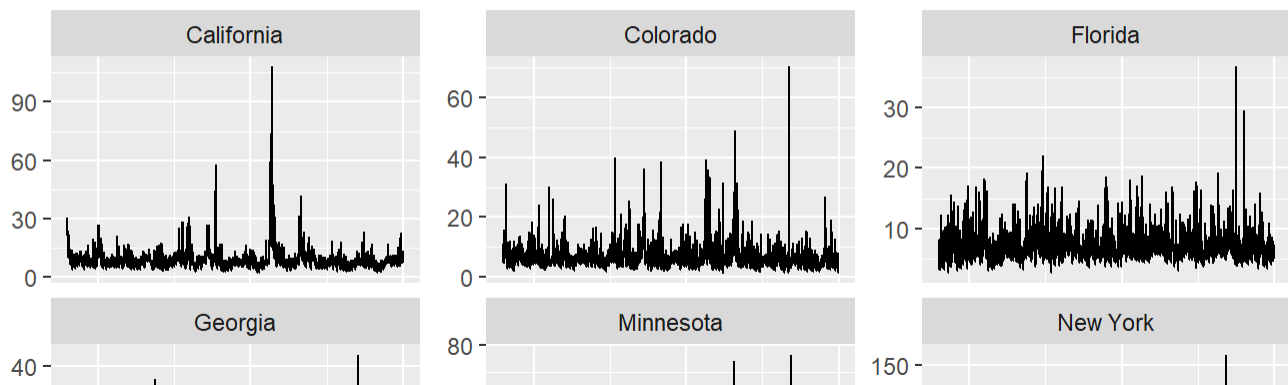
PM_{2.5} (µg/m³) – Daily Means by State

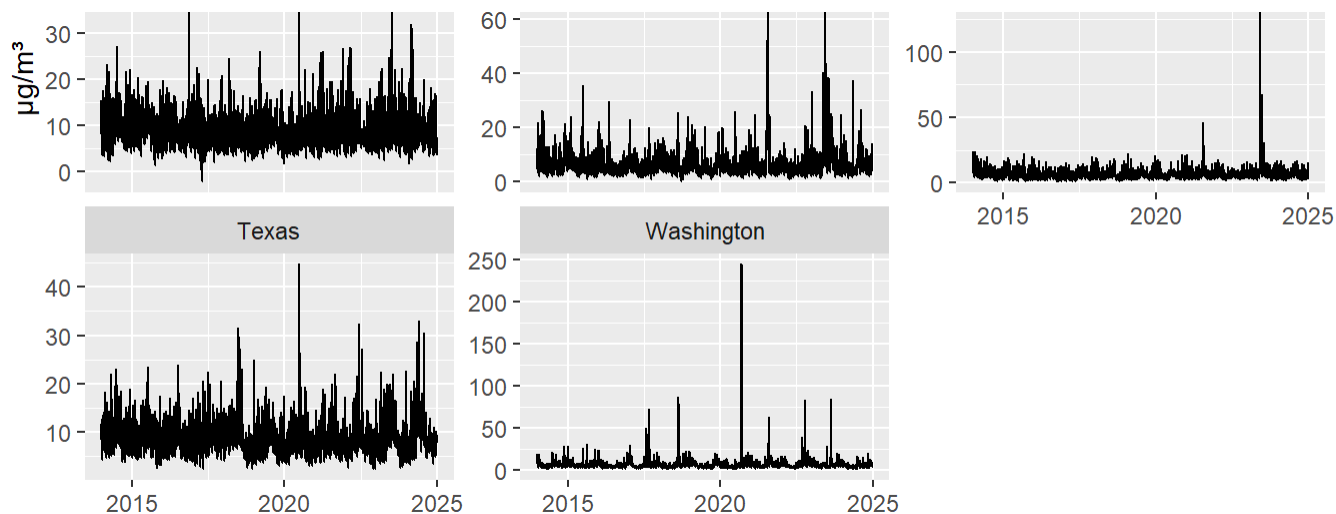
```

ggplot(merged_daily, aes(date, pm25_ugm3, group = state_name)) +
  geom_line() + facet_wrap(~ state_name, scales = "free_y") +
  labs(title = "PM2.5 (µg/m³), Daily Means by State", x = NULL, y = "µg/m³")

```

PM2.5 (µg/m³), Daily Means by State



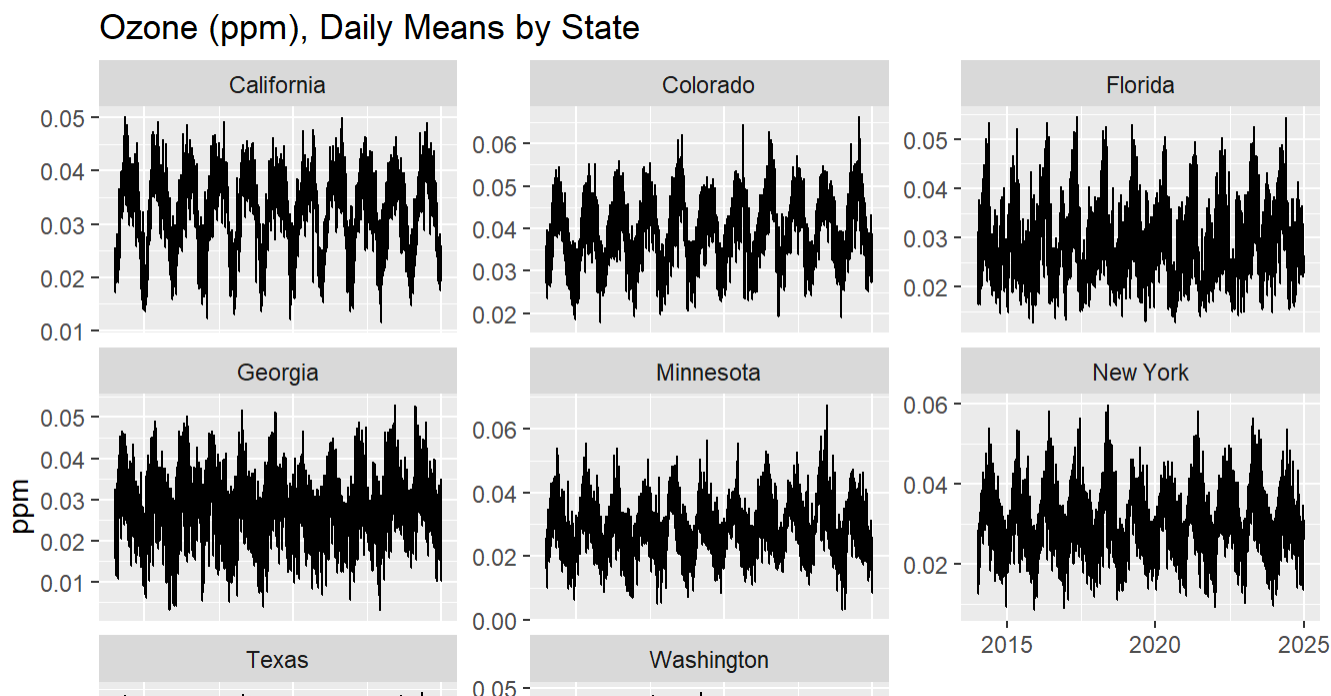


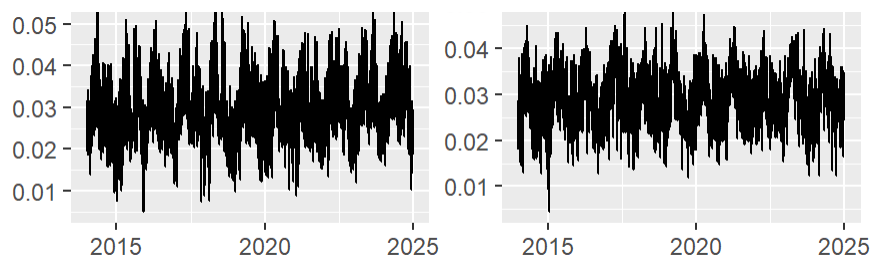
interpretation:

Across all states, PM_{2.5} concentrations exhibit strong seasonal cycles—typically peaking in winter months and dipping in summer. Western states (Washington, California, Colorado) show higher short-term volatility (which could be linked to wildfire events), while southeastern states (Georgia, Florida) remain comparatively stable with lower average levels.

Ozone (ppm) – Daily Means by State

```
ggplot(merged_daily, aes(date, ozone_ppm, group = state_name)) +
  geom_line() + facet_wrap(~ state_name, scales = "free_y") +
  labs(title = "Ozone (ppm), Daily Means by State", x = NULL, y = "ppm")
```



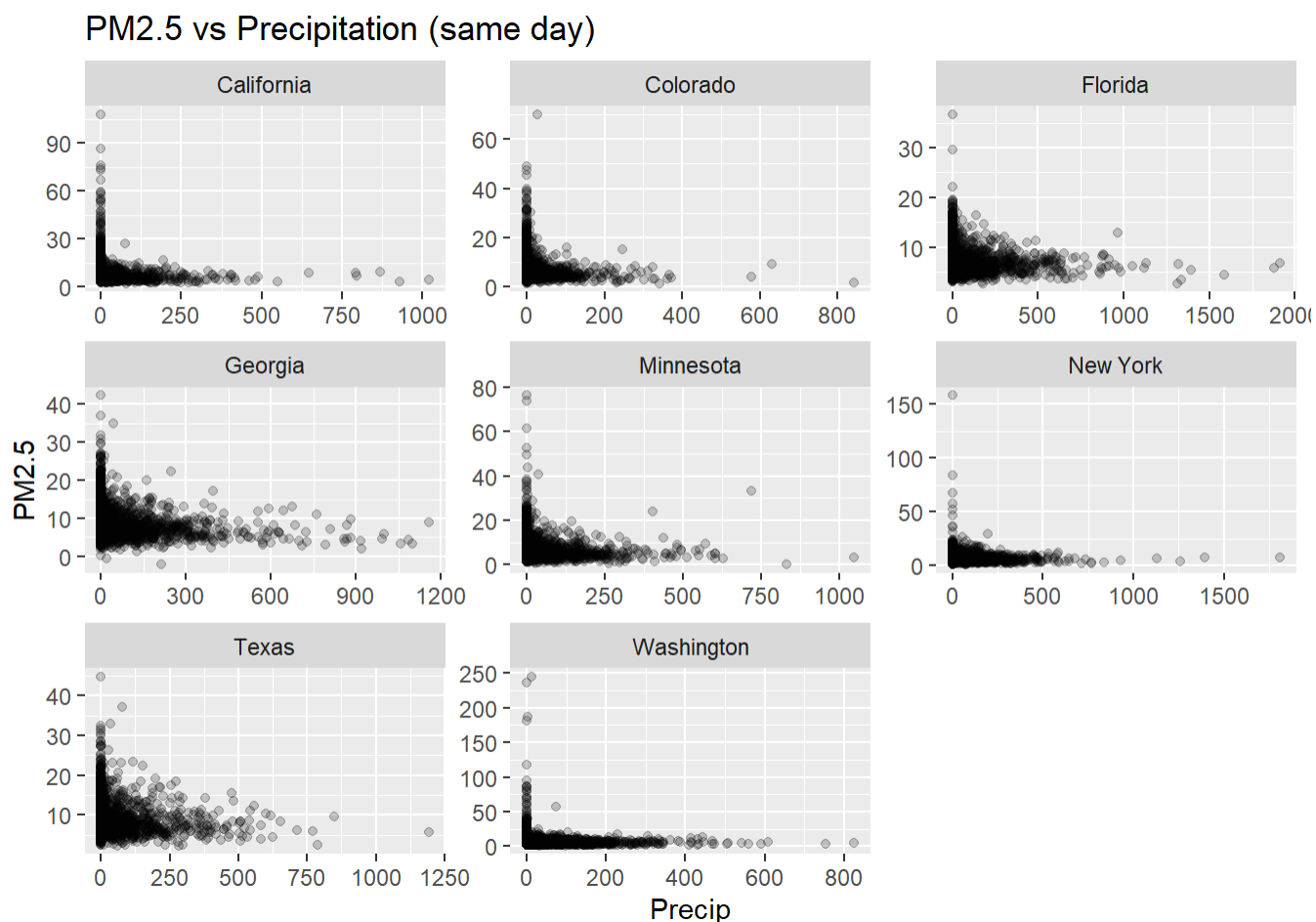


interpretation:

Ozone displays the opposite seasonal tendency, with higher concentrations during warmer months when photochemical reactions intensify. States with hotter climates such as Texas and Georgia exhibit prolonged summer peaks, while northern states (Minnesota, Washington) show shorter high-ozone seasons.

PM_{2.5} vs. Precipitation (same day)

```
ggplot(merged_daily, aes(prcp, pm25_ugm3)) +  
  geom_point(alpha = 0.2) + facet_wrap(~ state_name, scales = "free") +  
  labs(title = "PM2.5 vs Precipitation (same day)", x = "Precip", y = "PM2.5")
```

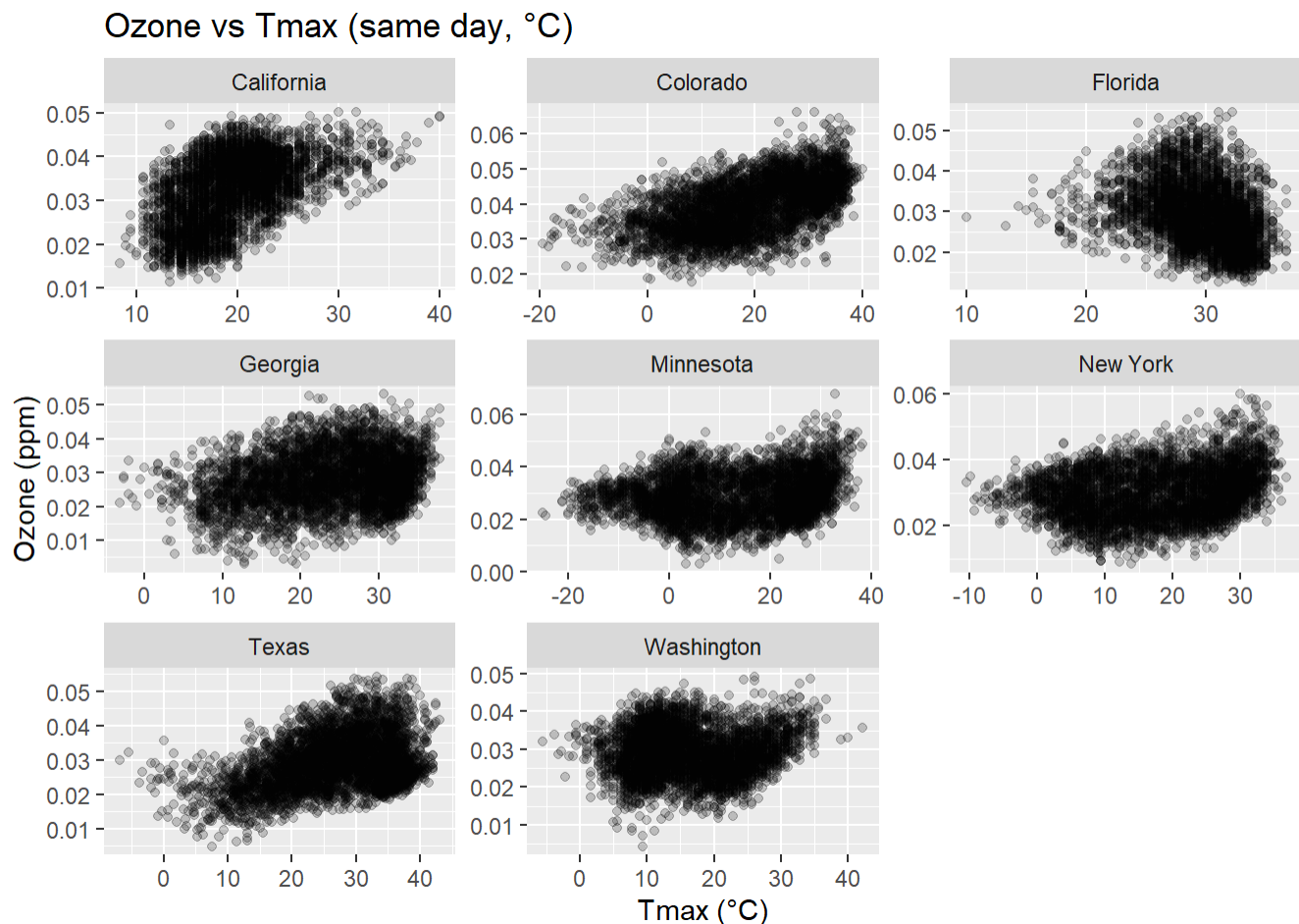


interpretation:

An inverse relationship is evident—days with measurable precipitation correspond to lower $\text{PM}_{2.5}$ concentrations due to wet-deposition effects. The relationship is strongest in humid regions (Florida, Georgia) and weakest in drier states (Colorado, California).

Ozone vs. Tmax (same day, °C)

```
ggplot(merged_daily, aes(tmax, ozone_ppm)) +  
  geom_point(alpha = 0.2) + facet_wrap(~ state_name, scales = "free") +  
  labs(title = "Ozone vs Tmax (same day, °C)", x = "Tmax (°C)", y = "Ozone (ppm)")
```



interpretation:

Ozone levels generally rise with increasing maximum temperature, consistent with photochemical formation processes. Warm, sunny conditions drive higher O_3 , especially in southern and central states

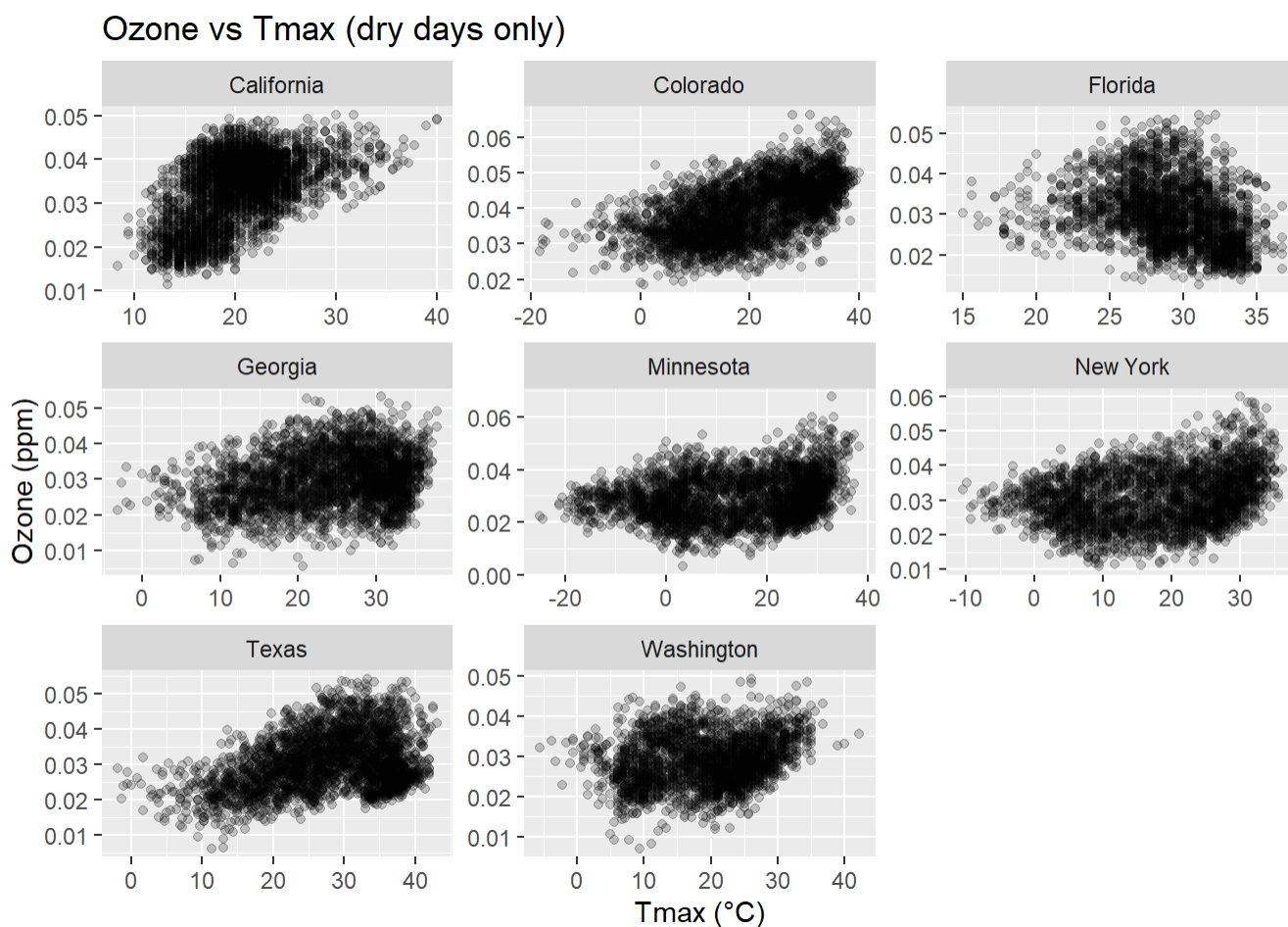
(Texas, Georgia), confirming temperature as a key predictor of ozone variability.

In contrast, Florida's scatter shows a weaker or slightly inverted same-day relationship. When filtered to dry or warm-season days, this anomaly lessens but does not disappear, suggesting that Florida's ozone dynamics are influenced by coastal meteorology and convective weather patterns not fully represented in this dataset.

appendix a - florida

referenced "dry" filtering

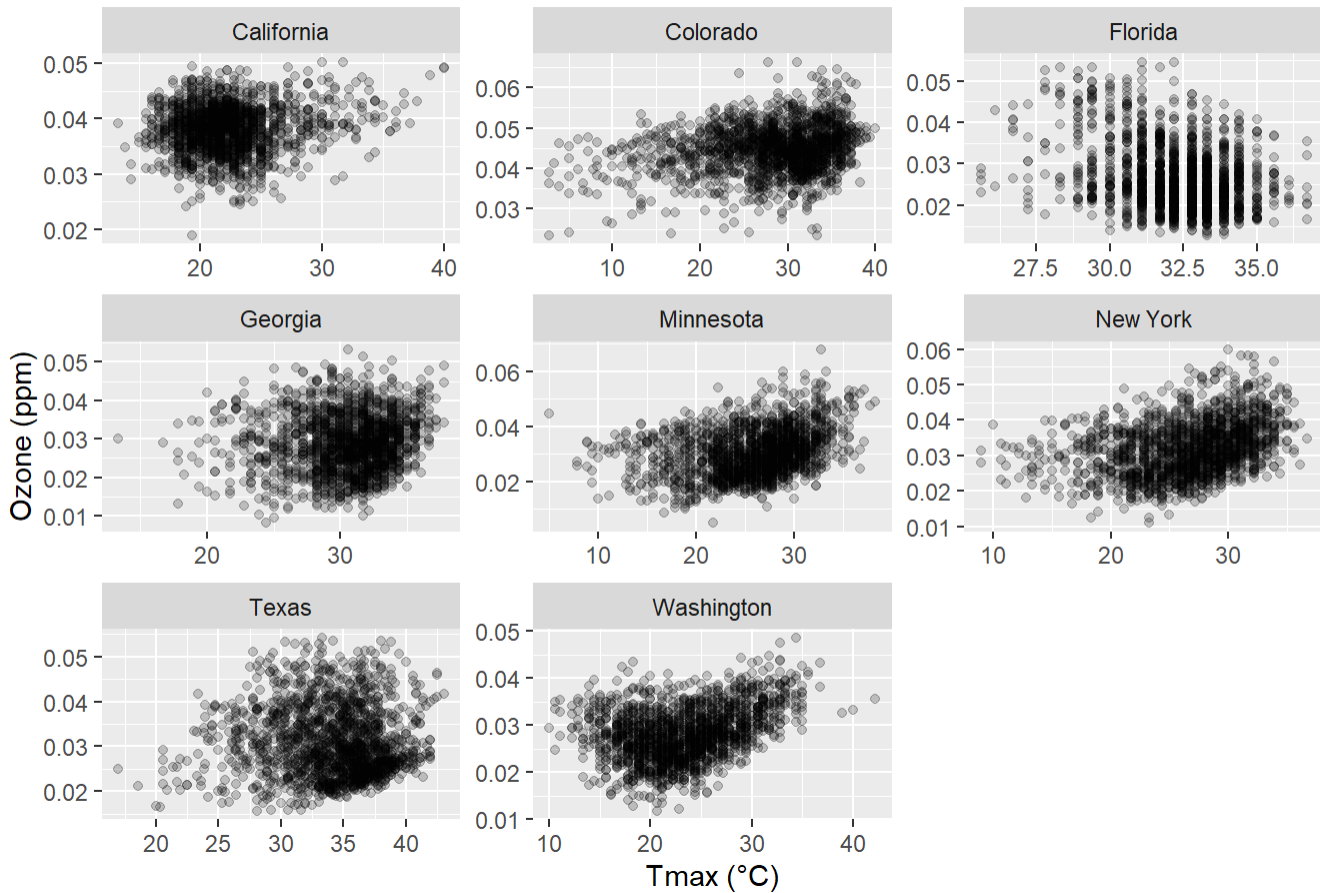
```
dry <- merged_daily |> dplyr::filter(is.na(prcp) | prcp == 0)
ggplot(dry, aes(tmax, ozone_ppm)) +
  geom_point(alpha = 0.2) + facet_wrap(~ state_name, scales = "free") +
  labs(title = "Ozone vs Tmax (dry days only)", x = "Tmax (°C)", y = "Ozone (ppm)")
```



referenced "warm" filtering

```
warm <- merged_daily |> dplyr::filter(lubridate::month(date) %in% 5:9)
ggplot(warm, aes(tmax, ozone_ppm)) +
  geom_point(alpha = 0.2) + facet_wrap(~ state_name, scales = "free") +
  labs(title = "Ozone vs Tmax (warm season only)", x = "Tmax (°C)", y = "Ozone (ppm)")
```

Ozone vs Tmax (warm season only)



appendix b - exploration

initial designs - regions

Early in development I considered categorizing states into broad geographic regions (West, Central, East). At first this was intended mainly as a way to demonstrate step 6 of the project guidelines — the conversion of categorical variables to factors — by creating an additional categorical field (region). In practice, the simpler approach of converting existing variables such as `state_name`, `parameter`, and `units` already satisfied that requirement without adding another layer of grouping. The regional framework remains a valid extension, however, and could still be used in future iterations to explore large-scale geographic contrasts in pollutant behavior.

interactivity of lessR library

If I were to revisit this project using that developmental version, I could imagine using lessR's interactive plotting functions to let viewers toggle among pollutants, seasons, or states, or to display multivariate summaries (for example, comparing PM_{2.5}, ozone, and temperature on a radar chart). That kind of interface would turn the current static report into an exploratory dashboard, bridging the reproducibility of scripted analysis with the accessibility of an interactive data story.